Adaptive Cruise Control Algorithm and System Modeling

Collaborator(s): Ruby Choi, Jorge Del Angel, Yahir Dominguez, Sam Herrera

Date Submitted: 2025 May 4

1 Abstract

The objective of this project is to create an ideal Adaptive Cruise Control capable of simulating safe driving within a two-driver scenario. In MATLAB, a Proportional Integral Derivative algorithm was created to control the follower car's acceleration to achieve the targeted gap of 100 ft (adjusted for different scenarios) starting from an initial position and velocity of 0 ft and 0 ft/s. Safety conditions, such as a max braking of 10 ft/s^2, were included. Euler's method was used to get an update to the car's position and velocity on short intervals. A passing feature was also included to overtake the lead car on command. In various scenarios, the PID parameters (Proportional term of 0.1, Derivative Term of 1, and Integral Term of .005) were changed to observe behavioral changes. With a constant lead car velocity, a high proportional term (0.9) caused jerky behavior due to overreactions, while a low proportional term (0.005) had sluggish motion from inefficient reactions. In another scenario where the lead car suddenly stopped, a high derivative term (6) led to excessive braking, while a low derivative term (0) caused a mild overshoot of the targeted gap.

2 Introduction

A modern variant on the long-existing cruise control technology, the Adaptive Cruise Control (ACC) is designed to refine driver safety through the use of onboard sensors and control algorithms to modulate acceleration and braking responses according to different conditions on the road. Like its predecessor, the ACC allows for longer drives without the need to press the throttle constantly, reducing driver fatigue substantially. However, as opposed to cruise control, ACC does more than just maintain a constant speed by continuously monitoring the vehicle's distance and speed relative to others and adjusting accordingly to maintain safe parameters[1].

In this project, the ACC system is simulated based on a student-developed MATLAB code. The MATLAB simulation models two vehicles — one defined as a lead car with defined speed parameters and the other a follower car that dynamically adjusts its velocity based on a fixed distance between the two. A control algorithm — specifically the PID (Proportional Integral Derivative) controller — is used to control the output to regulate the process value to a set point[2]. In other words, it controls the follower car's throttle (acceleration) to maintain the vehicle at a target distance from the lead car. Beyond the objective's scope, the MATLAB code also contains several instances of visualizations and quantifications of the system behavior through data tables and an animation of the two-vehicle system. Furthermore, the simulation contains

additional scenarios such as the follower car passing the lead car by pressing the key ‘p’, stop and go traffic, and highway merging. This behavior is also visualized in the animation. Moreover, graphs of the following car’s acceleration and derivative of the error gap are provided.

3 Methods

A 1-dimensional MATLAB-based simulation was developed to model an Adaptive Cruise Control (ACC) system using a simplified two-vehicle setup. The system consisted of a lead vehicle with a prescribed velocity profile and a following vehicle controlled by a PID algorithm. Real-time measurement of the distance between vehicles was assumed. Simulation logic was implemented using Euler’s Method to update vehicle states at each time step.

3.1 Lead vehicle

The lead vehicle was modeled with an initial position and a time-dependent velocity profile to match and simulate realistic traffic conditions. The lead car began at a fixed position 100 feet ahead of the follower car and followed a piecewise-defined velocity function of the following factors:

- A constant speed of 50 mph
- Emergency braking to a stop
- Deceleration and acceleration between the speeds of 60 and 75 mph
- A brief stop phase that simulates a red light in a traffic stop
- A gradual acceleration returning to a highway speed of 60 mph

Alternate scenarios such as emergency braking, constant speed, and stop-and-go behaviors were defined within the code and could be toggled for desired testing. These scenarios include linear acceleration and deceleration, such as maximum braking rates of near 10 ft/s^2 and acceleration rates of up to 7.5 ft/s^2. The lead car was not reactive to the follower car in any way,

3.2 Follower car

The follower vehicle begins at x-position of 0 feet (except for the scenario of smooth acceleration and deceleration of the lead car) with an initial speed of zero and was programmed to pursue a target speed equivalent to 70 mph (converted to feet per second). The control objective was to maintain a desired distance of 100 feet behind the lead vehicle, a value consistent with adaptive cruise control settings[4]. This gap remained constant throughout the simulation for simplicity.

The follower car adjusted its velocity using an adaptive control scheme driven by the real-time gap between it and the lead vehicle. If the gap exceeded the target, the car attempted to approach its maximum speed. If it fell below the desired threshold, the car adjusted to match or reduce speed relative to the lead vehicle, ensuring collision avoidance.

3.2.1 Adaptive algorithm

To regulate the follower car's motion, a Proportional-Derivative-Integral (PID) controller was implemented. The controller modulated the vehicle's acceleration based on the gap error:

- error_gap(i) = gap_actual - gap_target;
- dError = (error_gap(i) - error_gap(i-1)) / h;
- int_error = int_error + 0.5 * (error_gap(i) + error_gap(i-1)) * h;

Where *error_gap(i)* is the proportional term, *dError* is the derivative of the error, and *int_error* accumulates the integral of the error using the trapezoidal rule

The PID control law was expressed as:

- error_vec = [error_gap(i); dError; int_error];
- acceleration(i) = K' * error_vec;

Euler's method was used for numerical integration of the vehicle dynamics, which allowed for straightforward updates of position and velocity at each time step. This method was selected for its simplicity and adequacy given the short time step used (0.05 seconds). The integral error term was computed using trapezoidal integration for improved stability.

The controller included additional logic to reduce noise and avoid overreacting to small fluctuations in the error signal. When both the gap error and its derivative were below a certain threshold, the acceleration was instead approximated using the derivative of the lead car's velocity, effectively reducing unnecessary control adjustments.

Acceleration was saturated within physical limits (−10 ft/s² to +7.5 ft/s²), and the car's speed was constrained not to exceed its target speed. Safety checks ensured the follower slowed down if the lead car was too close or decelerating rapidly.

3.3 Visualization and Analysis

A simulation was made to visually show the results of the code in real-time. First, a simulation of the lead (blue) and following (red) car was run to show the different scenarios depending on which was desired. Next, plots of both cars' speeds, the gap between them, and the follower cars' acceleration would be created. Finally, error dynamics (the derivative of the gap error) would be produced to evaluate the controller's performance.

4 Results

The following scenarios are simulated through the MATLAB code, assuming a highway speed of 70 mph. Starting at a distance larger than that of the target gap, the lead car travels at constant speed with the follower car following behind at a greater velocity. The follower car adjusts its speed—by accelerating, decelerating, or braking—to maintain a safe following distance (80 or 20 depending on the scenario) as the lead car changes speed or stops suddenly by applying heavy braking. Initial parameter values are set as: time step of 0.05, proportional term of 0.1, derivative term of 1, and an integral term of 0.005. The time step of 0.05 was determined to be

small enough for the following scenarios for relatively quick computation and accurate recording of the simulation behavior. The following scenarios also investigated the effects of changing the terms on the following car behavior.

Figure 1 shows a plot of the first scenario in which the lead car starts ahead of the follower car at a distance larger than that of the target gap, travelling at a constant speed while the follower car accelerates to a target speed greater than that of the lead car. It decelerates at the target gap of 80 ft behind the lead car to a matching speed. The simulation time was also cut down to 25 seconds after determining it to be enough time to judge the results properly.

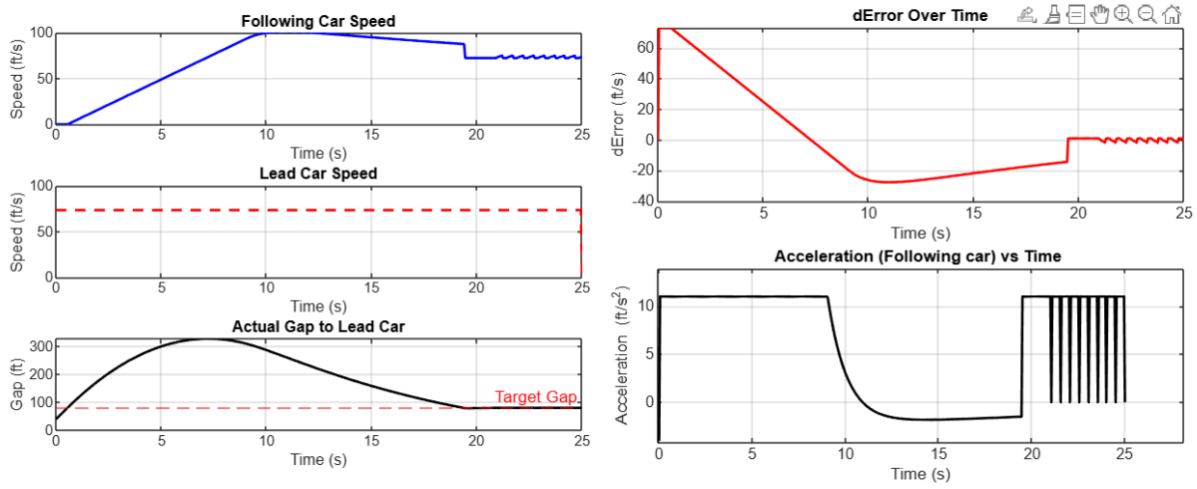


Figure 1 – A plot of the lead car speed, follower car speed, and gap between the two vehicles. Plots measuring error over time and acceleration over time are also shown. The lead car starts 40 ft ahead going a constant speed of 50 mph, whereas the follower car starts from rest and has a target speed of 70 mph, decelerating at a target gap of 80 ft behind the lead car.

Figures 2 and 3 show the same scenario when the proportional term is set too high and too low, respectively 0.9 and 0.005 while the derivative term and integral term were kept constant at 1 and 0.005.

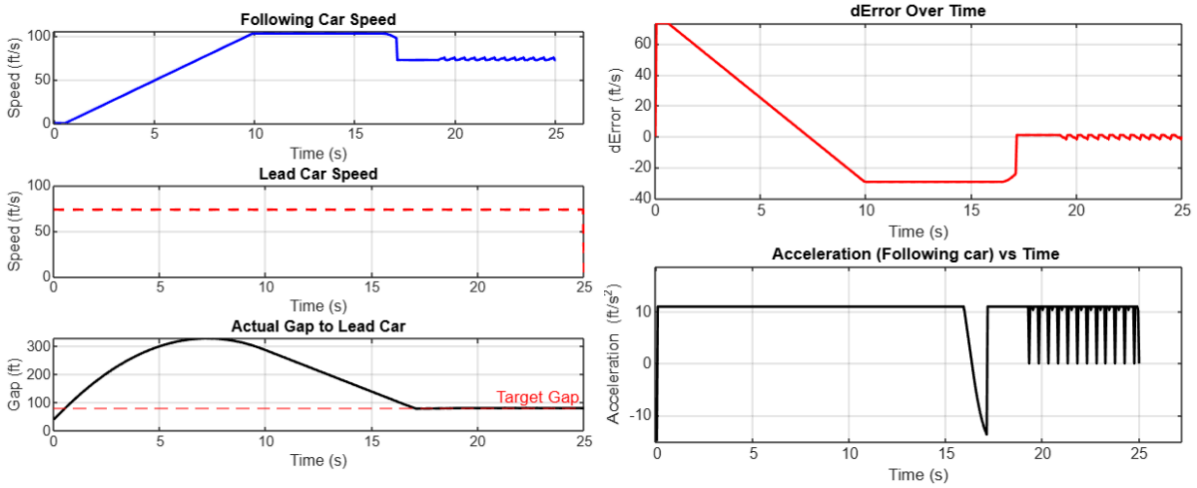


Figure 2 – Proportional term is set to 0.9.

The following car overreacts when it reaches the target gap, suddenly coming to a stop;

essentially it demonstrates the behavior of jerky driving implying potential instability.

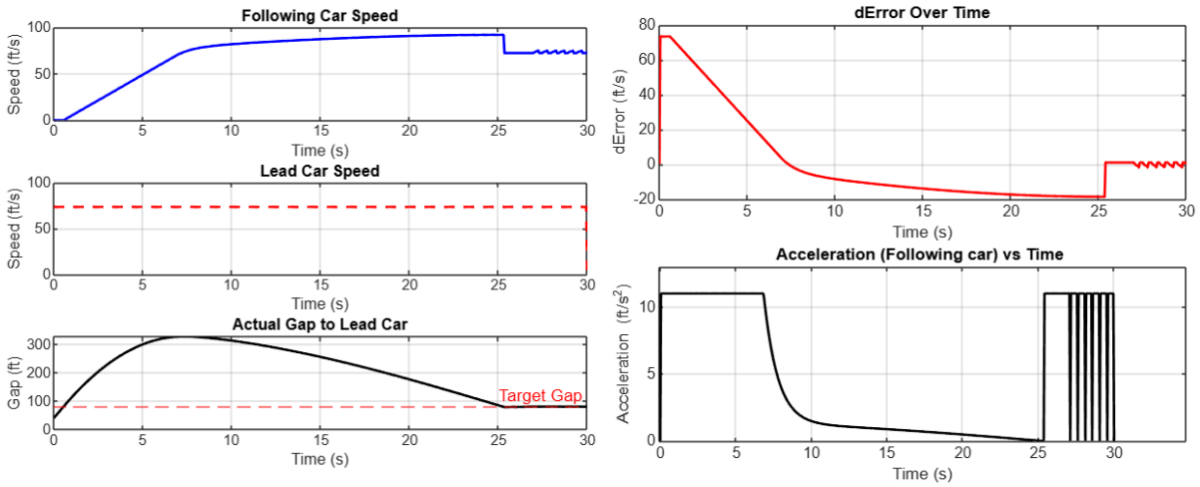


Figure 3 – Proportional term is set to 0.005.

When the proportional term is too low, the following car responds sluggishly and takes far longer than necessary to reach the target distance, denoting its inefficiency.

Figure 4 shows a plot of the second scenario in which the follower car matches the speed of at the target gap of 80 ft behind the lead car when it suddenly stops.

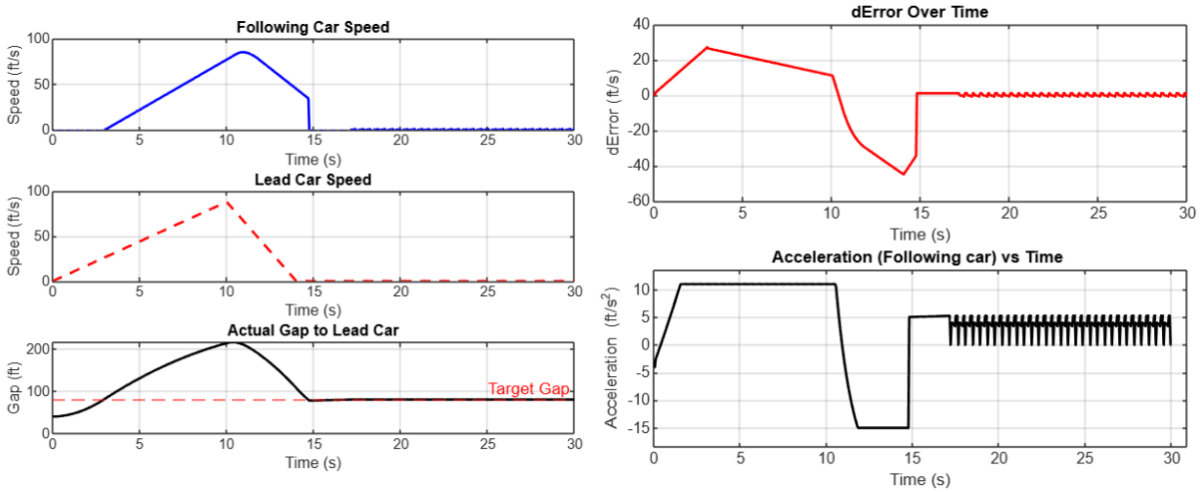


Figure 4 – A plot of the lead car speed, follower car speed, and gap between the two vehicles. Plots measuring error over time and acceleration over time are also shown. The lead car starts 40 ft ahead accelerating to 50 mph before applying heavy braking to come to a stop as the follower car starts from rest and adjusts its acceleration to stop at the target gap of 80 ft behind the lead car.

Figures 5 and 6 show the same scenario when the derivative term is set too high and too low, respectively 6 and 0 while the proportional term and integral term were kept constant at 0.1 and 0.005.

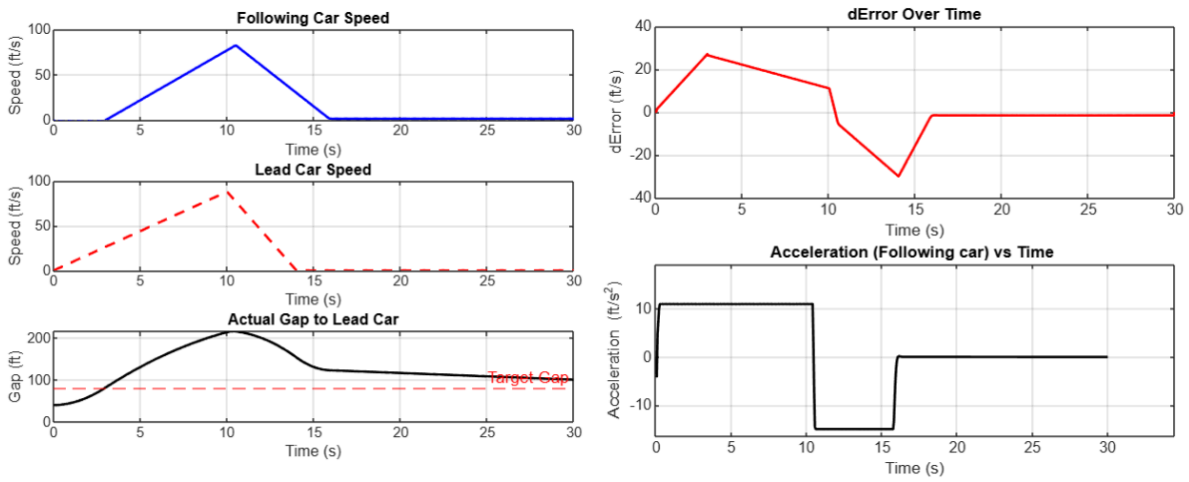


Figure 5 – Derivative term is set to 6.

The following car applies excessive braking or decelerates (indicated by the plot of acceleration over time) as the lead car comes to a stop. Demonstrating the behavior of unsafe braking, the behavior of the system when the derivative term is too high is proven to be unstable. Furthermore, the following car fails to reach the target gap in the simulation time.

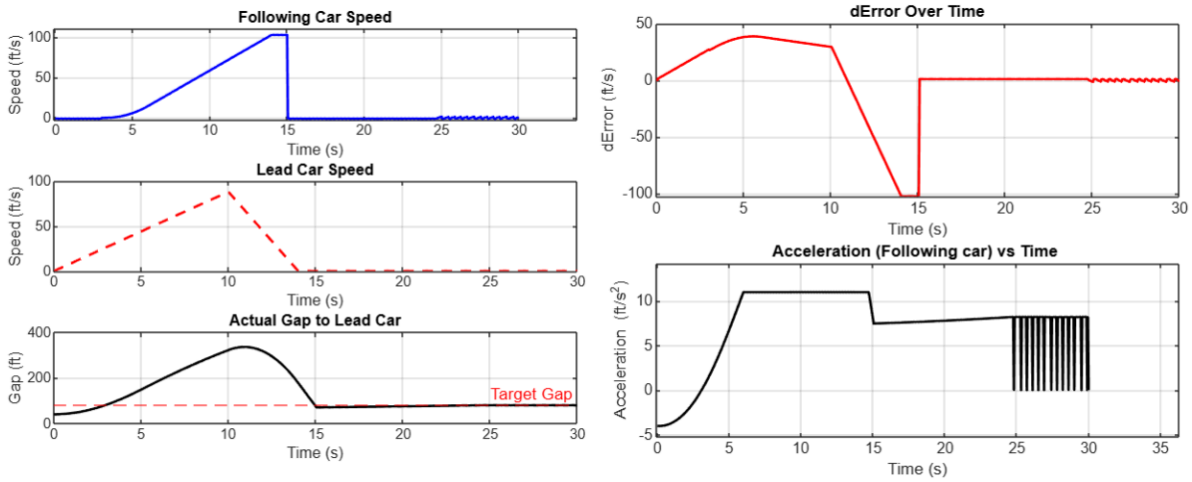


Figure 6 – Derivative term is set to 0.

The following car ends up slightly overshooting the target gap from the lead car before correcting itself.

Figure 7 shows a plot of an additional scenario based on the previous situation in which the follower car passes the lead car with the press of the key ‘p’, accelerating to its target speed as it does so.

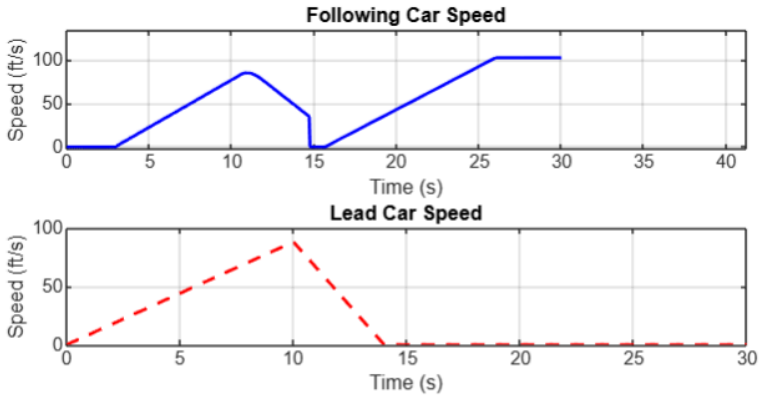


Figure 7 – A plot of the lead car speed and follower car speed. Plots measuring error over time and acceleration over time are also shown. The lead car accelerates then brakes heavily with the following car slowing down behind the lead car before passing and accelerating to its target speed.

Figure 8 shows a plot of the third scenario in which the follower car matches the speed of the lead car as it smoothly accelerates then decelerates over a range of speeds (determined to reasonably be 60-75 mph based on the highway speed limits across the nation[3]).

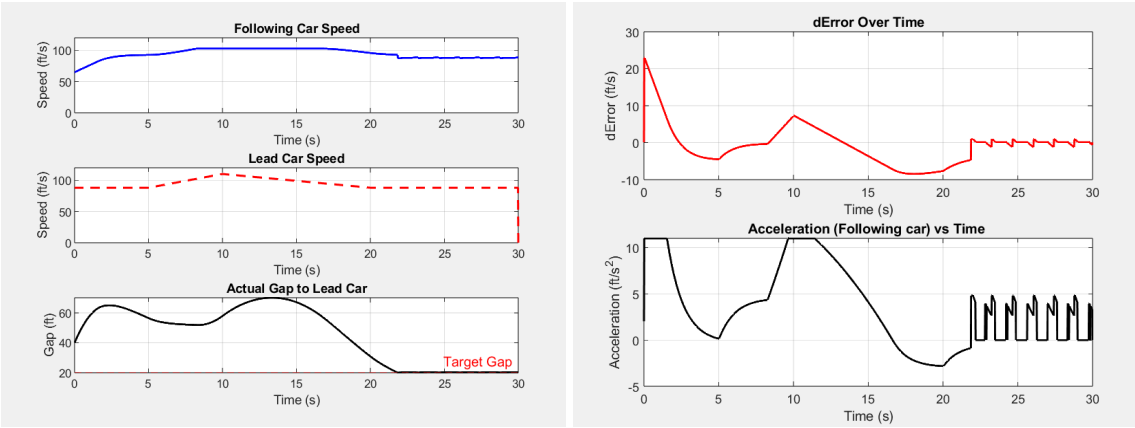


Figure 8 – A plot of the lead car speed, follower car speed, and gap between the two vehicles. Plots measuring error over time and acceleration over time are also shown. The lead car smoothly accelerates and decelerates over a range of speeds with the follower matching the changing speeds.

Figures 9 and 10 show the same scenario when the integral term is set too high and too low, respectively 1.5 and 0 while the proportional term and derivative term were kept constant at 0.1 and 1.

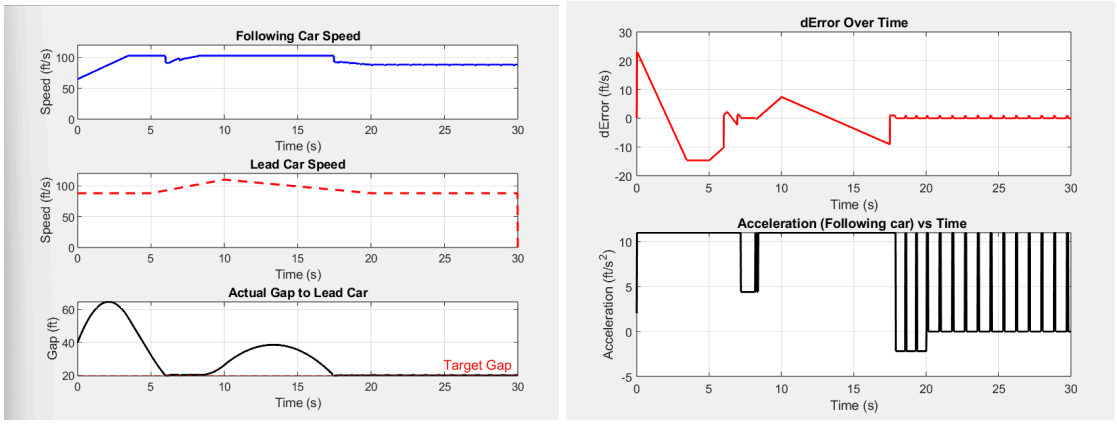


Figure 9 – Integral term is set to 1.5.

The oscillation of the following car becomes tighter and more frequent, indicating the vehicle is overcorrecting and is thus, unstable.

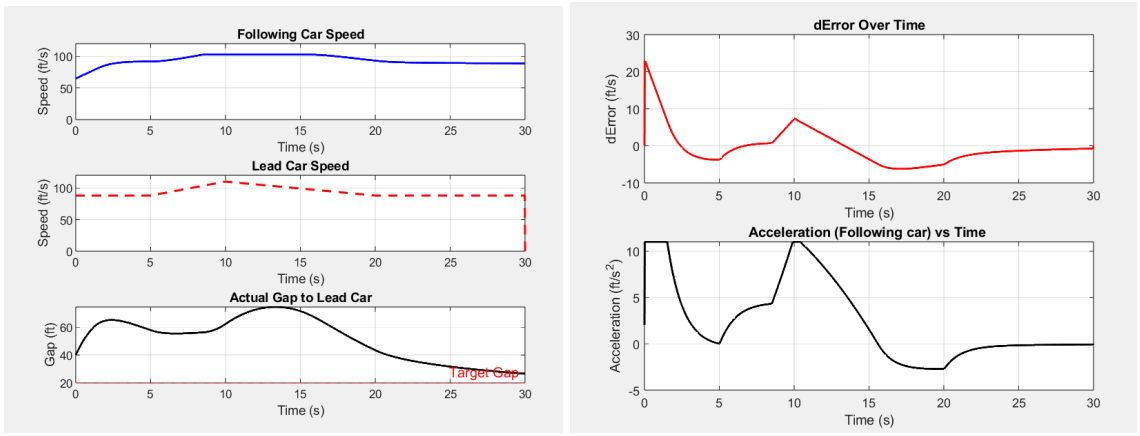


Figure 10 – Integral term is set to 0.

The following car demonstrates similar behavior to initial integral term parameters, but lacks the noise that the integral term introduces into dError and acceleration of the following car.

Figure 11 shows a scenario beyond the objective's scope modeling two vehicles stopping at a traffic light before merging into a highway. The target gap was adjusted to 20 ft to accurately model the stopping distance between vehicles at a traffic stop.

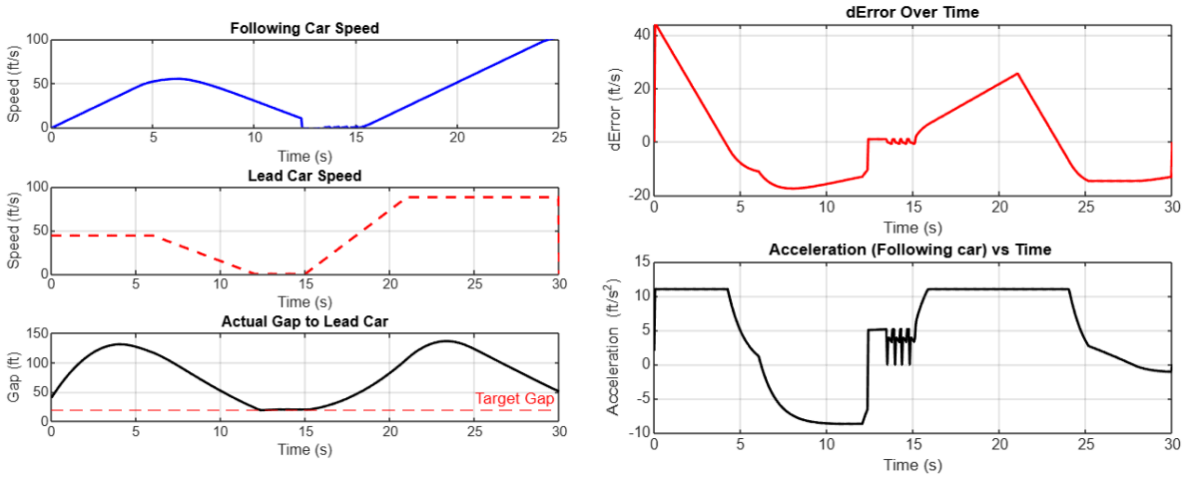


Figure 11 – A plot of the lead car speed, follower car speed, and gap between the two vehicles. Plots measuring error over time and acceleration over time are also shown. The lead car smoothly decelerates to a stop at a traffic light, followed soon after by the following car before both vehicles accelerate to merge into the highway with the following car accelerating to its target speed.

Figure 12 shows a scenario beyond the objective's scope modeling two vehicles continuously stopping and going during traffic on a highway. The target gap was adjusted to 20 ft to accurately model the stopping distance between vehicles during traffic.

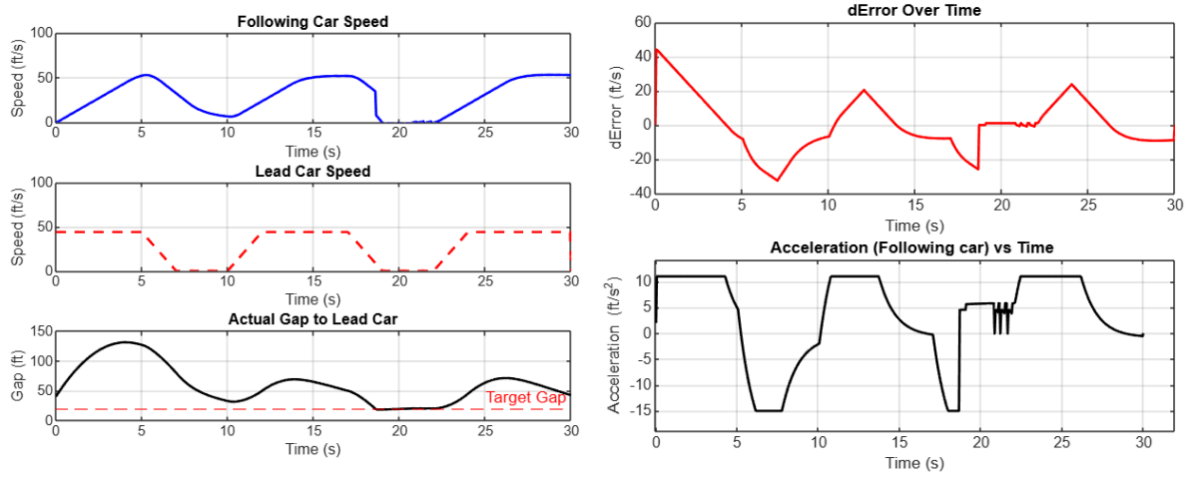


Figure 12 – A plot of the lead car speed, follower car speed, and gap between the two vehicles. Plots measuring error over time and acceleration over time are also shown. The lead car smoothly decelerates to a stop and accelerates to a cruising speed at 30 mi/hr, continuously stopping and going while the following car adjusts its speed accordingly to maintain a target gap of 20 feet.

5 Discussion

The results of the MATLAB simulation confirm the effectiveness of the follower car's response to the lead car across various scenarios—so long as control parameters were tuned based on the situation—including smooth acceleration, deceleration, emergency braking, and passing by lane switching.

As for the parameters themselves, even though each parameter has a different function in the PID controller, there were consistent trends regarding high or low terms. Higher terms like an Ideal Proportional of 0.9 or a Derivative Term of 6 lead the car to overreact more often. This overreaction is bad in the sense that the driver becomes more unstable and unsafe due to hard acceleration changes (hard braking). On the contrary, lower terms, such as a 0 in the Derivative Term, cause the follower car to react late. This late response could lead to much higher gaps between the follower car and the lead car in the beginning, but also cause the car to not react in time to be within the safe gap limit when accelerating to reach the lead car.

Sources of error that may be responsible for potential discrepancies in the plot include: numerical errors in which large time steps may cause inaccuracies in vehicle speeds, and modeling errors where the simulation may assume a lack of friction or drag. The system closely follows the real world, accounting for how two vehicles would interact in a two-lane road but fails to reflect more dynamically complex scenarios in which traffic may heavily affect how the follower car will respond. For example, the follower car — in order to pass the lead car — would have to account for other passing vehicles before switching lanes.

To expand on this model, there is potential in incorporating sensors to simulate noise and latence to further improve driver safety by warning them of potential threats, introducing obstacle detection. Furthermore, there may be room for multi-vehicle scenarios, creating more realistic dynamics between vehicles in the model to observe the effects of chain reactions in traffic.

6 Conclusion

Overall, the PID's parameters for the ACC need to be within an optimal range to ensure that the ACC can work in an efficient and safe manner. Safety should be a top priority within these simulations to ensure that the outcomes are close to reality.

7 References

[1] W. Clarke, "What Is Adaptive Cruise Control?," [Online]. Available: https://cars.usnews.com/cars-trucks/advice/what-is-adaptive-cruise-control. [Accessed 05 29 2025].

[2] PID Explained, "PID Controller Explained," [Online]. Available: https://pidexplained.com/pid-controller-explained/. [Accessed 05 29 2025].

[3] P. Novak, "USA Driving Guide – Important Rules, Speed Limits & Signs," [Online]. Available: https://wikiusa.org/en/driving-in-the-usa. [Accessed 05 29 2025].

[4] "2022 Tundra - Dynamic radar cruise control with full-speed range*" [Online]. Available: https://www.toyota.com/owners/warranty-owners-manuals/digital/article/tundra/2022/om0c029u/ch04se050405/. [Accessed 05 29 2025].

8 Appendix

The MATLAB code for the two-vehicle ACC simulation is shown below.

```
%% Numerical Methods Project - Adaptive Cruise Control Simulation
clear; close all; clc;
%% User-Defined Parameters
max_speed_mph = 70;                 % Maximum speed of following car [mi/hr]
conversion = 5280 / 3600;           % Conversion factor [mi/hr to ft/s]
target_speed = max_speed_mph * conversion;  % Target speed in [ft/s]
gap_target = 100;                    % Desired following distance [ft]
h = 0.05;                           % Time step [s]
t_end = 30;                         % Total simulation time [s]
t = 0:h:t_end;                      % Time vector
K = [0.1, 1, .005]';                 % PID gains [Kp, Kd, Ki]
% flag for passing behavior
p = 0;
global pass_car;
pass_car = false;
% Key press callback to trigger passing
function passEvent(src, event)
   global pass_car
   if strcmp(event.Key, 'p')       % Press 'p' to initiate passing
       pass_car = true;
   end
end
%% Initialization
x_following = zeros(1, length(t));   % Position of following car [ft]
v_following = zeros(1, length(t));   % Velocity of following car [ft/s]
acceleration = zeros(1, length(t));  % Acceleration of following car [ft/s²]
x_lead = zeros(1, length(t));        % Position of lead car [ft]
v_lead = zeros(1, length(t));        % Velocity of lead car [ft/s]
x_lead(1) = 40;                      % Initial position of lead car [ft]
error_gap = zeros(1, length(t));     % Error in gap [ft]
int_error = 0;                       % Integral of gap error
dError = zeros(1,length(t));
%% Lead Car Velocity Profile (Traffic light to highway merge)
% v_lead_func = @(time) ...
%     (time < 6) .* (30 ) + ...
%     (time >= 6 & time < 12) .* max(30 -  5* (time - 6),0)  + ...
%     (time >= 12 & time < 15) .* 0 + ...
%     (time >= 15 & time < 21) .* ((10 * (time - 15)) ) + ...
%     (time >= 21) .* (60 );
```

```
%Stop and go traffic
% v_lead_func = @(time) ...
%     (time < 5) .* (30 ) + ...                                  % cruising at 30
mi/hr
%     (time >= 5 & time < 7) .* ((30 - 15 * (time - 5)) ) + ... % decelerate to 0
%     (time >= 7 & time < 10) .* 0 + ...                                    %
stopped for 3 seconds
%     (time >= 10 & time < 12) .* ((15 * (time - 10)) ) + ...   % accelerate back
up
%     (time >= 12 & time < 17) .* (30 ) + ...                   % cruise again
%     (time >= 17 & time < 19) .* ((30 - 15 * (time - 17)) ) + ... % slow down
again
%     (time >= 19 & time < 22) .* 0 + ...                                   %
stopped again
%     (time >= 22 & time < 24) .* ((15 * (time - 22)) ) + ...   % accelerate again
%     (time >= 24) .* (30 );                                    % back to cruising
%Constant speed
% v_lead_func = @(time) 50;  % constant 50 mi/hr
%Emergency Breaking
% v_lead_func = @(time) (time < 10) .* (6*time) + (time >= 10 & time < 14) .*max(60
- 15 * (time - 10),0)  + (time >= 9) .* (0 );  % emergency break from 50 mi/hr
%Smooth Acceleration and deceleration
v_lead_func = @(time) (time<5).*60+(time>=5 & time <
10).*(60+3*(time-5))+(time>=10&time<20).*(75-1.5*(time-10))+(time>=20).*60;  %
decelerating and accelerating
v_following(1)=65;
%% Animation Setup
fig = figure;
set(fig, 'KeyPressFcn', @passEvent);
xlim([0, 1500]); ylim([0, 0.2]);
title('Car Animation');
hold on;
Following_car = plot([0, 10], [0, 0], 'r-', 'LineWidth', 16);  % Red: Following car
Lead_car = plot([x_lead(1), x_lead(1)+10], [0, 0], 'b-', 'LineWidth', 16);  % Blue:
Lead car
hold off;
%% Simulation Loop
for i = 1:length(t)-1
   if pass_car == false
       % Lead car motion
       v_lead(i) = v_lead_func(t(i))*conversion; %unit conversion
       x_lead(i+1) = x_lead(i) + v_lead(i) * h;
       % Gap error calculations
       gap_actual = x_lead(i) - x_following(i);
       error_gap(i) = gap_actual - gap_target;
       % PID error terms
       if i == 1
           dError(i) = 0;
       else
           dError(i) = (error_gap(i) - error_gap(i-1)) / h;
           int_error = int_error + 0.5 * (error_gap(i) + error_gap(i-1)) * h;
```

```
        end
        % PID controller output
        error_vec = [error_gap(i); dError(i); int_error];
        acceleration(i) = K' * error_vec; %[ft/s^2]
        % Noise reduction
        if i > 1 && abs(error_gap(i)) < 0.2 && abs(dError(i)) < 0.2
            acceleration(i) = (v_lead(i) - v_lead(i-1)) / h;
        end
        % Acceleration limits
        acceleration(i) = max(min(acceleration(i), 60 * conversion / 8), -15);
%researched reasonable acceleration
        % Velocity update
        v_following(i+1) = v_following(i) + acceleration(i) * h; %ft/s   Euler's
Method
        % Speed logic based on gap
        if gap_actual > gap_target
            v_following(i+1) = max(min(v_following(i+1), target_speed), 0);
        else
            v_following(i+1) = min(v_following(i), v_lead(i) - 1);
        end
        % Position update
        x_following(i+1) = x_following(i) + v_following(i) * h;
        % Animate cars
        xlim([x_following(i) - 50, x_following(i) + 200]);
        set(Following_car, 'XData', [x_following(i)-10, x_following(i)]);
        set(Lead_car, 'XData', [x_lead(i), x_lead(i)+10]);
        drawnow;

    else
        % Passing behavior
        v_lead(i) = v_lead_func(t(i))*conversion;
        x_lead(i+1) = x_lead(i) + v_lead(i) * h;
        acceleration(i) = 10;
        v_following(i+1) = v_following(i) + acceleration(i) * h;
        v_following(i+1) = max(min(v_following(i+1), target_speed), 0);
        x_following(i+1) = x_following(i) + v_following(i) * h;
        % Animate passing motion
        p = p + h / 50;
        xlim([x_following(i) - 50, x_following(i) + 200]);
        set(Following_car, 'XData', [x_following(i)-10, x_following(i)], 'YData',
[min(0.02,p), min(0.02,p)]);
        set(Lead_car, 'XData', [x_lead(i), x_lead(i)+10]);
        drawnow;
    end
end
%% Plot Results
gap = x_lead - x_following;  % Final gap
figure;
subplot(3,1,1);
plot(t, v_following, 'b', 'LineWidth', 1.5);
xlabel('Time (s)');
```

```
ylabel('Speed (ft/s)');
title('Following Car Speed');
ylim([0 120]);
grid on;
subplot(3,1,2);
plot(t, v_lead, 'r--', 'LineWidth', 1.5);
xlabel('Time (s)');
ylabel('Speed (ft/s)');
title('Lead Car Speed');
ylim([0 120]);
grid on;
if pass_car == false
   subplot(3,1,3);
   plot(t, gap, 'k', 'LineWidth', 1.5); hold on;
   yline(gap_target, 'r--', 'Target Gap');
   xlabel('Time (s)');
   ylabel('Gap (ft)');
   title('Actual Gap to Lead Car');
   grid on;
%% PID Error Dynamics
figure;
subplot(2,1,1);
plot(t, dError, 'r', 'LineWidth', 1.5);
xlabel('Time (s)');
ylabel('dError (ft/s)');
title('dError Over Time');
grid on;
subplot(2,1,2)
plot(t, acceleration, 'k', 'LineWidth', 1.5); hold on;
xlabel('Time (s)');
ylabel('Acceleration (ft/s^2)');
title('Acceleration (Following car) vs Time ');
grid on;
hold off;
end
```